

10.3.2 SLAM 建图与地图管理

课程信息

- 环境：Windows 10、WSL2 Ubuntu 24.04、ROS 2 Jazzy、Gazebo Harmonic。
- 本节目标：用 SLAM Toolbox 完成三房间建图，保存 PGM/YAML 地图对，并做地图质量分析。
- 主路径：Gazebo + RViz 手动/半自动建图。
- 兜底路径：轻量 2D 仿真 + CP2 无界面验收。

路径约定：本讲义假设项目位于 `~/mecamind_ros2`。若路径不同，请自行替换。建议先 `git fetch --tags` 且 `git checkout lesson2.0`（第二节课 tag 发布后），或使用老师指定的当前分支。

本节不要求完成 Nav2 导航、YOLO 或语音任务。导航会在下一次课使用本节保存的地图。

1. 本节学习目标

完成本节后，应当能够：

1. 说清 SLAM 的数据流：激光、里程计、TF、仿真时钟如何汇入建图节点。
2. 解释 `map` → `odom` → `base_footprint` 坐标系关系，以及 `use_sim_time` 为何必须开启。
3. 用统一入口 `mapping.launch.py` 在 Gazebo 与轻量仿真上建图。
4. 完成手动遥控建图与半自动路线建图，并保存地图。
5. 阅读地图质量报告（覆盖率、未知区、自由区连通性、墙体完整性）。
6. 比较 SLAM Toolbox、Cartographer、RTAB-Map 的适用边界。
7. 使用 `scripts/test_cp2.sh` 完成 CP2 验收。

2. 本节重点内容

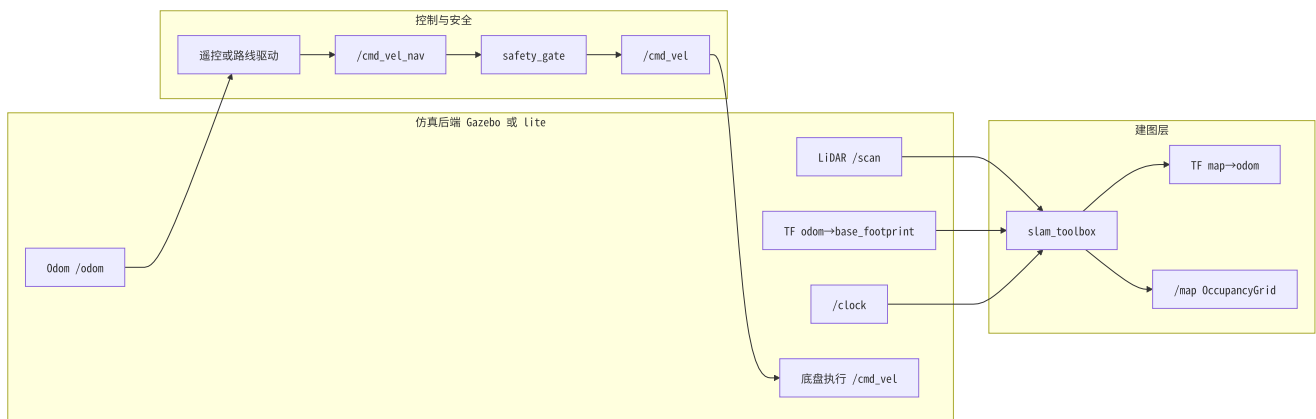
2.1 SLAM 在课程中的位置

第一节课打通了仿真层接口：`/scan`、`/odom`、`/cmd_vel`、`/tf`、`/clock`。

第二节课在这些接口之上接入 **SLAM Toolbox**，把未知环境扫成一张二维占据栅格地图（OccupancyGrid）。

地图是后续 Nav2 的输入资产。没有合格地图，定位、规划和巡航都无从谈起。

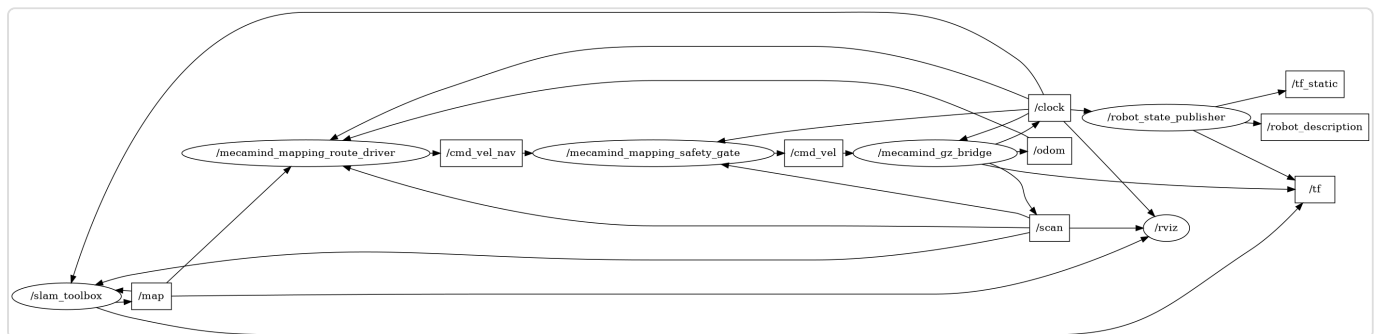
2.2 建图数据流



要点：

- **传感器输入**： `/scan` + 里程计。注意： `slam_toolbox` 读取里程计用的是 `TF odom → base_footprint`，而不是直接订阅 `/odom` 话题（两者内容一致，都由仿真发布）。
- **时间同步**：所有节点 `use_sim_time:=true`，订阅 `/clock`，否则会出现 TF 外推失败或空地图。
- **SLAM 输出**： `/map`（栅格地图）以及 `map → odom` 变换。
- **速度链路**：遥控或路线驱动发到 `/cmd_vel_nav`，经 `safety_gate` 碰撞检查后输出 `/cmd_vel`，两个后端都只消费 `/cmd_vel`。

下面是建图运行时实测的节点/话题连接图（Gazebo 后端，已隐去 rqt、参数事件等无关节点；椭圆是节点，方框是话题）：



对照上图从右往左读两条链：

1. **控制链**：`mecamind_mapping_route_driver`（半自动时；手动时换成键盘遥控）发 `/cmd_vel_nav` → `mecamind_mapping_safety_gate` 检查激光距离后发 `/cmd_vel` → `mecamind_gz_bridge` 送进 Gazebo 驱动底盘。
2. **感知链**：`mecamind_gz_bridge` 发布 `/scan`、`/odom`、`/clock`、`/tf`；`slam_toolbox` 订阅 `/scan`、`/clock` 和 `/tf`，输出 `/map` 和 `map → odom`（也走 `/tf`）；RViz 订阅 `/map`、`/scan` 做可视化。

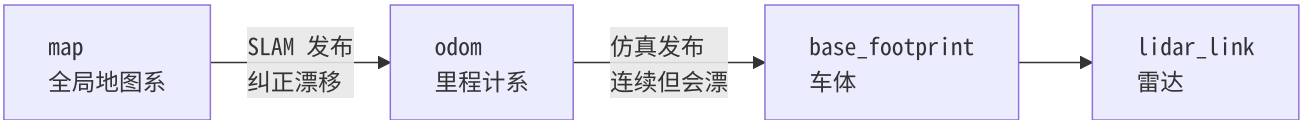
课堂上可以让学员自己跑 `rqt_graph`（视图切到 "Nodes/Topics (active)"）对照验证。

2.3 TF: 为什么要有 map 和 odom

坐标系	含义
map	全局地图坐标系；SLAM 通过闭环修正漂移后，发布 map → odom
odom	里程计坐标系；连续但会漂移
base_footprint	机器人在地面上的投影中心

第一节课只有 odom → base_footprint。建图后多出 map → odom：短时运动靠里程计，长时间靠激光匹配把全局位姿“拉回”地图。

TF 链框图（从全局到车体）：



对照记忆：

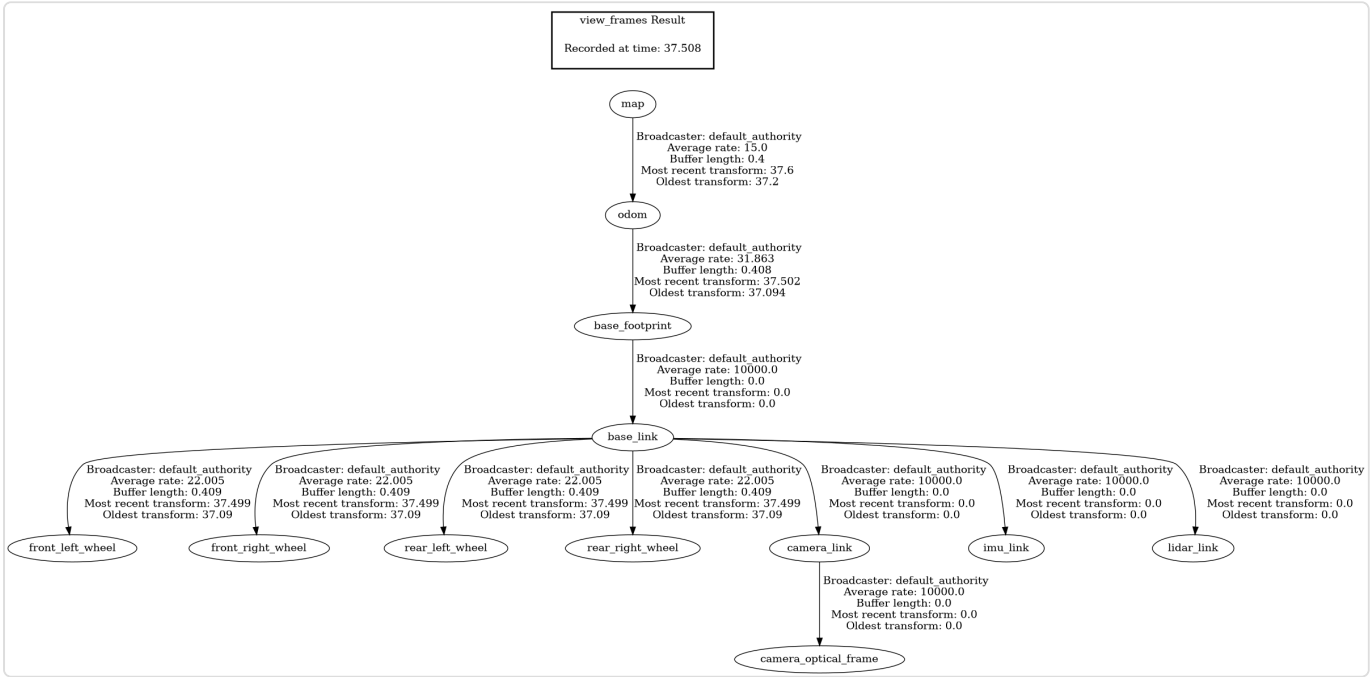
第一节课：

odom → base_footprint → ... → lidar_link

本节课补上：

map → odom → base_footprint → ... → lidar_link

建图运行时用 `ros2 run tf2_tools view_frames --ros-args -p use_sim_time:=true` 导出的实测 TF 树如下：



读图要点（课堂可提问学员）：

- 最顶层 map → odom 由 SLAM 发布（图中频率约 15 Hz）——这条边第一节课是没有的。
- odom → base_footprint 由仿真发布（约 30 Hz），是里程计的 TF 形式。

- `base_footprint` → `base_link` → 四个轮子/`lidar_link`/`imu_link`/`camera_link` 来自 `robot_state_publisher` 读 URDF：轮子会转所以带时间戳持续更新，`lidar_link` 等固定安装件是静态变换（rate 显示 10000 即 static）。
- 激光帧 `frame_id` 是 `lidar_link`，SLAM 通过整条链把每束激光换算到 `map` 坐标系里再"画"进栅格。

`/odom` 里有什么、怎么起作用

`/odom` 的消息类型是 `nav_msgs/Odometry`。它不靠激光，主要靠轮子转了多少，估计「车现在在哪、速度多少」。可看成**短时较准、长时会漂的计步器**。

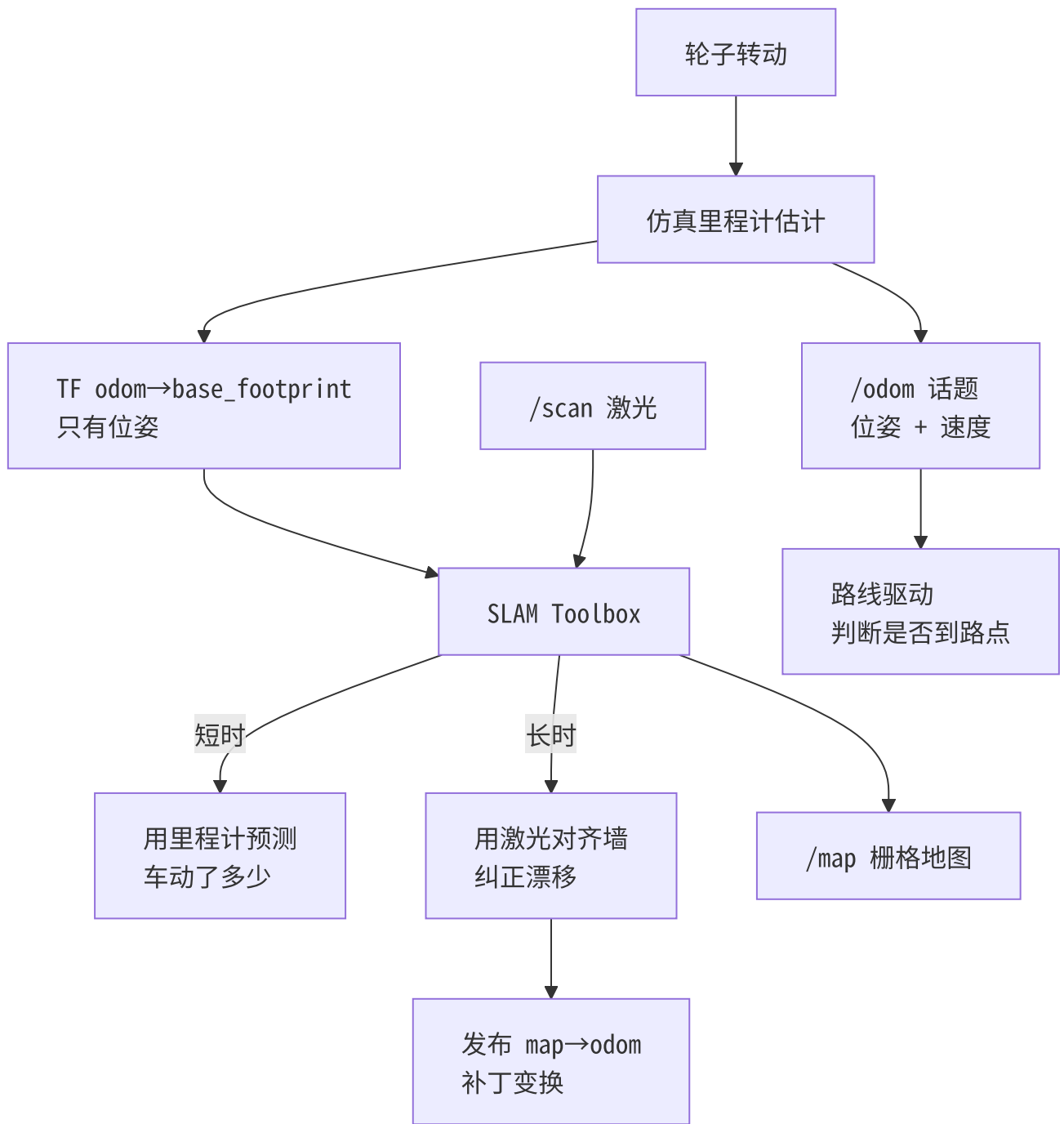
消息里最重要的两块：

字段	内容	含义
<code>pose.pose.position</code>	x, y (z 地面车多为 0)	车在 <code>odom</code> 坐标系里的位置
<code>pose.pose.orientation</code>	四元数（可换成偏航角 yaw）	车头朝向
<code>twist.twist.linear</code>	x, y	前后、横向速度
<code>twist.twist.angular</code>	z	原地转动角速度

另外还有时间戳 `header.stamp`、坐标系名（通常 `frame_id=odom`，`child_frame_id=base_footprint`）。仿真发 `/odom` 的同时，也发 TF `odom → base_footprint`，**两者表达同一份里程计信息，只是格式不同**：

- `/odom` 话题：带速度信息，适合节点直接订阅（路线驱动就用它判断到没到路点）。
- TF `odom → base_footprint`：只有位姿，走 TF 树，`slam_toolbox` 用的是这条（实测 `ros2 topic info -v /odom` 可见 SLAM 并不订阅 `/odom` 话题）。

`odom` 在建图里怎么起作用：



谁在用：

使用者	用哪种形式	作用
SLAM Toolbox	TF odom → base_footprint	两帧激光之间，先用里程计猜测车动了多少，再用激光对齐墙面纠正误差
半自动路线驱动	/odom 话题	读当前位置，判断是否到达路点、下一步往哪开
RViz / 调试	两种都可	观察车是否在动、朝向是否合理

建图时的分工可以记两句：

- **短距离**：多信 odom（连续、平滑，适合预测）。
- **长时间**：信激光匹配（打滑和误差会让 odom 慢慢漂）。SLAM 不直接改掉 odom，而是额外发布 map → odom，把漂掉的那一截补回来。

启动建图后可查看一帧：

```
ros2 topic echo /odom --once
```

重点看 pose.pose.position.x/y 是否随车移动变化。

2.4 双后端同一接口

mecamind_bringup/launch/mapping.launch.py 用 backend:=gazebo|lite 切换仿真后端。

换后端时，下面这些话题名、消息类型、TF 关系保持不变，所以 SLAM、遥控、安全门、存图脚本都不用改。

可以记成两条流水线：

1. **感知上行（给 SLAM 用）**：仿真发布 /scan、/odom、odom→base_footprint、/clock → slam_toolbox → 输出 /map 和 map→odom。
2. **控制下行（开车）**：键盘/路线驱动发 /cmd_vel_nav → safety_gate 做碰撞检查 → 输出 /cmd_vel → 仿真底盘执行。

接口	消息/内容	谁发布	谁订阅 / 干什么用
/scan	sensor_msgs/LaserScan（一圈激光距离）	Gazebo 或 lite 仿真	SLAM、安全门：看见墙在哪
/odom	nav_msgs/Odometry（轮式估计的位姿与速度）	仿真	路线驱动：判断到没到路点（SLAM 用的是它的 TF 形式 odom→base_footprint）
/cmd_vel_nav	geometry_msgs/Twist（期望速度）	遥控或路线驱动	安全门入口；手动遥控必须 remap 到这里
/cmd_vel	geometry_msgs/Twist（实际执行速度）	安全门	仿真底盘：真正让车动起来
TF	odom → base_footprint（第一节就有）	仿真	激光/车体在里程计坐标系里的位置
TF	map → odom（本节 SLAM 新增）	slam_toolbox	用激光匹配纠正里程计长期漂移
/clock	roscpp_msgs/Clock（仿真时间）	仿真	所有 use_sim_time:=true 的节点统一对时

注意区分两个速度话题：

- /cmd_vel_nav：你“想怎么开”（可能贴墙、过快，会被拦）。
- /cmd_vel：安全门放行后的速度；两个后端都只听这个开车。

课堂优先 Gazebo；机器卡顿或无 GUI 时用 lite。接口相同，所以 CP2 验收标准也相同。

2.5 手动建图 vs 半自动建图

模式	启动方式	谁开车	何时存图
手动	<code>auto_route:=false</code> （默认）	你用键盘遥控	你主动调用 <code>map_saver_cli</code> 存图
半自动	<code>auto_route:=true</code>	<code>mapping_route_driver</code> 按路点巡航	路线结束后自动保存到 <code>maps/</code>

半自动适合验收与重复实验；手动适合理解激光如何“刷”出墙体与房间。

2.6 地图文件格式

ROS 导航常用一对文件：

- `<名称>.pgm`：栅格图像（黑=占据，白=自由，灰=未知）。
- `<名称>.yaml`：分辨率、原点、占用阈值等元数据。

本项目默认保存到仓库内 `maps/` 目录（相对路径），例如：

```
maps/mecamind_three_room_map.pgm
maps/mecamind_three_room_map.yaml
```

生成物默认被 `.gitignore` 忽略；验收合格、要留给下节课用的地图可用 `git add -f` 强制纳入版本库。

2.7 地图质量分析看什么

`mecamind_map_quality_analyzer` 对照三房间世界几何，报告：

- **coverage**：感兴趣区域内已观测（自由+占据）比例。
- **unknown_ratio**：未知栅格比例。
- **largest_free_component_ratio**：最大连通自由区占比（是否“房间被墙封死/撕裂”）。
- **missing_walls**：外墙采样是否明显缺失。
- **verdict**：如 `commercial_ready`、`navigation_ready_best_effort`、`needs_remap`。

课堂验收要求 **verdict** 至少达到 `navigation_ready_best_effort`；CP2 脚本按此判定。

2.8 SLAM Toolbox / Cartographer / RTAB-Map 怎么选

方案	特点	本课为何用
SLAM Toolbox	2D 激光、生命周期节点、与 Nav2 生态契合、参数易调	默认方案：三房间 2D 建图足够
Cartographer	Google 出品，2D/3D，子图与全局优化强，配置与依赖更重	大场景/多传感器融合时可对比，非本课主线
RTAB-Map	视觉/RGB-D 为主，可带激光；算力与标定要求更高	后续有深度相机时再考虑

本课场景是平面激光 + 室内平面地图，选 SLAM Toolbox 最直接。

3. 启动前准备

```
cd ~/mecamind_ros2
bash scripts/cleanup_ros2.sh # 若上节课仿真未关干净
bash scripts/build.sh
source /opt/ros/jazzy/setup.bash
source ~/mecamind_ros2/install/setup.bash
```

WSLg 下若 RViz/Gazebo 报 display 错误：

```
export DISPLAY=:0
export WAYLAND_DISPLAY=wayland-0
```

4. LAB 1: 手动建图（Gazebo + RViz）

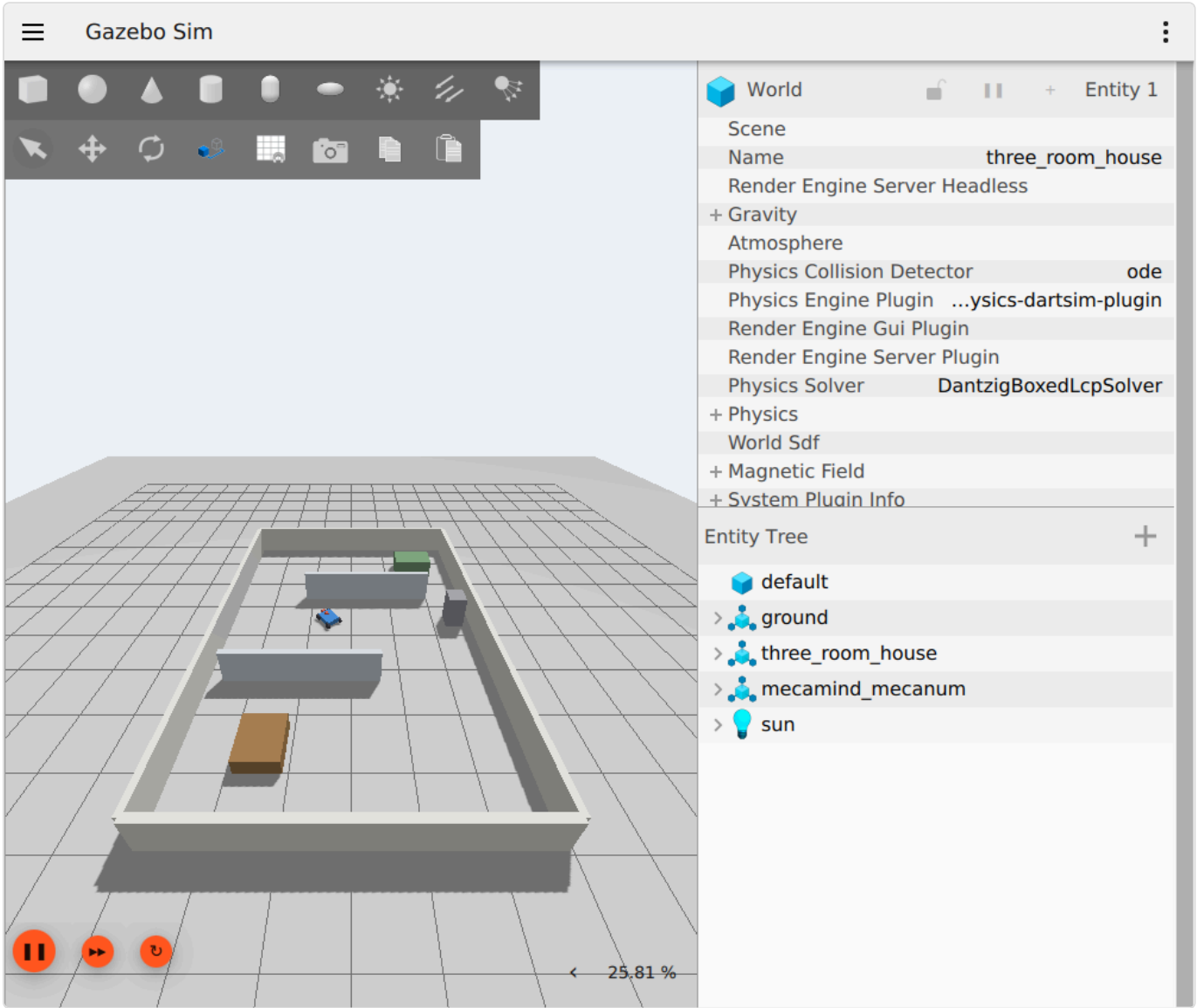
终端 1 —— 启动建图（默认后端 Gazebo，开 GUI 与 RViz）：

```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch mecamind_bringup mapping.launch.py
```

常用参数：

参数	默认值	作用
backend	gazebo	gazebo 或 lite
use_gui	true	是否打开 Gazebo 窗口
with_rviz	true	是否打开建图 RViz
auto_route	false	是否启用半自动路线
map_save_path	maps/mecamind_three_room_map	地图保存前缀（相对项目/启动目录）

启动成功后，Gazebo 窗口应显示三房间世界与机器人（机器人出生在地图原点附近，Entity Tree 中能看到 three_room_house 与 mecamind_mecanum）：



终端 2 —— 遥控（注意发到安全门入口）：

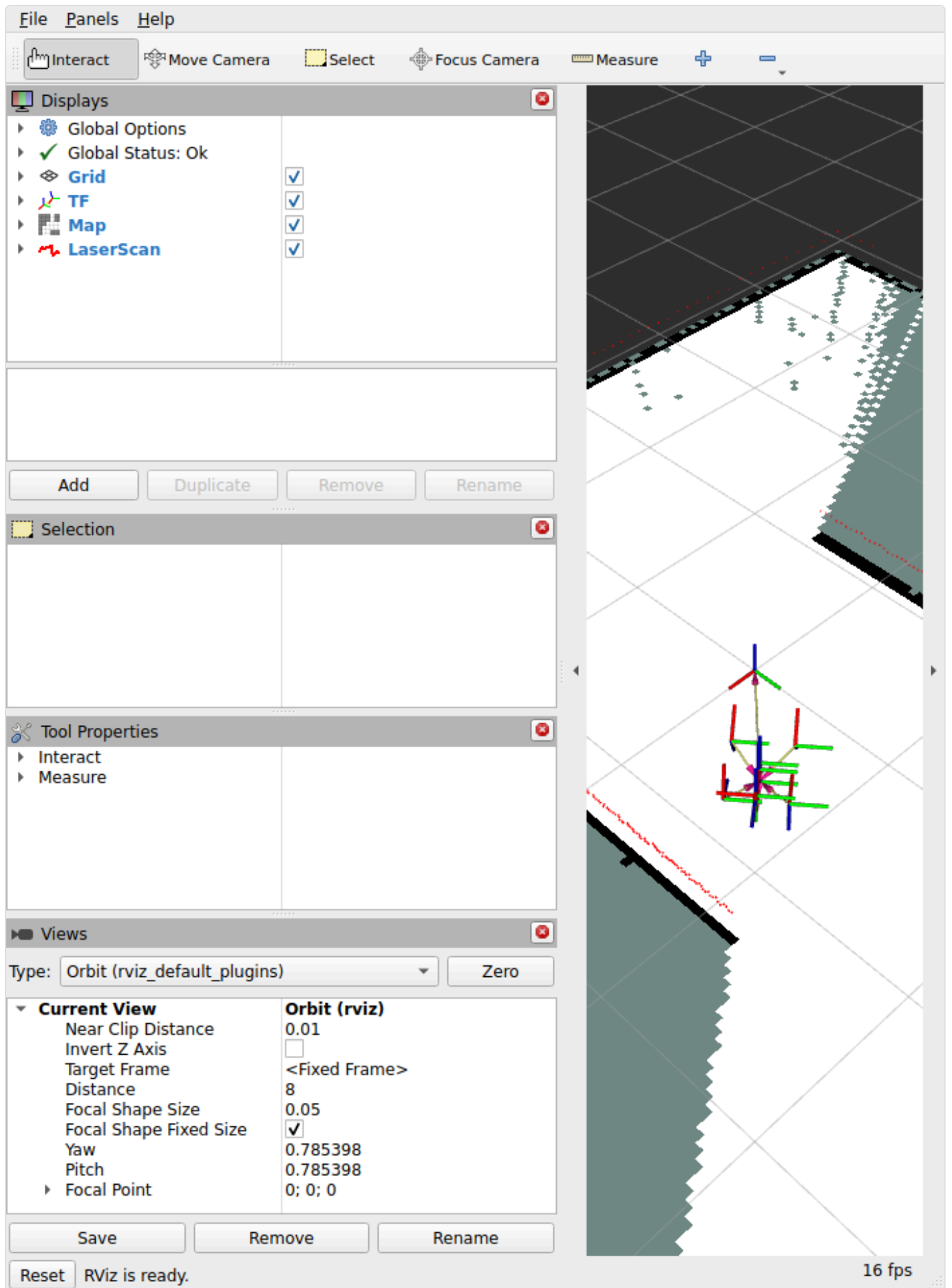
```
source /opt/ros/jazzy/setup.bash
source ~/mecamind_ros2/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r /cmd_vel:=/cmd_vel_nav
```

建议线速度约 0.2 m/s 、角速度约 0.5 rad/s 。缓慢驶过客厅、走廊、卧室，让激光多扫墙面与门口；避免高速冲撞导致安全门长时间刹停。

RViz 中应看到：

- Fixed Frame 为 `map`。
- `/map` 栅格逐渐填满。
- `/scan` 激光贴墙。
- 机器人位姿随运动更新。

建图初期的 RViz 画面像这样——白色是已确认的自由区，黑色是墙，深灰色是还没扫到的未知区；红色小点是当前激光打在墙上的位置，中间的坐标轴是机器人 TF：



给学员指出三件事：(1) 机器人一动，白色区域就跟着"刷"开；(2) 激光点始终贴在黑色墙边上，说明扫描匹配正常；(3) 如果地图完全空白，先检查 Fixed Frame 是否为 `map`、`use_sim_time` 是否开启（见第 9 节排错）。

存图（路线结束后也可用同一命令）：

```
cd ~/mecamind_ros2
ros2 run nav2_map_server map_saver_cli -f maps/mecamind_three_room_map \
  --ros-args -p map_subscribe_transient_local:=true -p use_sim_time:=true
```

确认生成：

```
ls -la maps/mecamind_three_room_map.pgm maps/mecamind_three_room_map.yaml
```

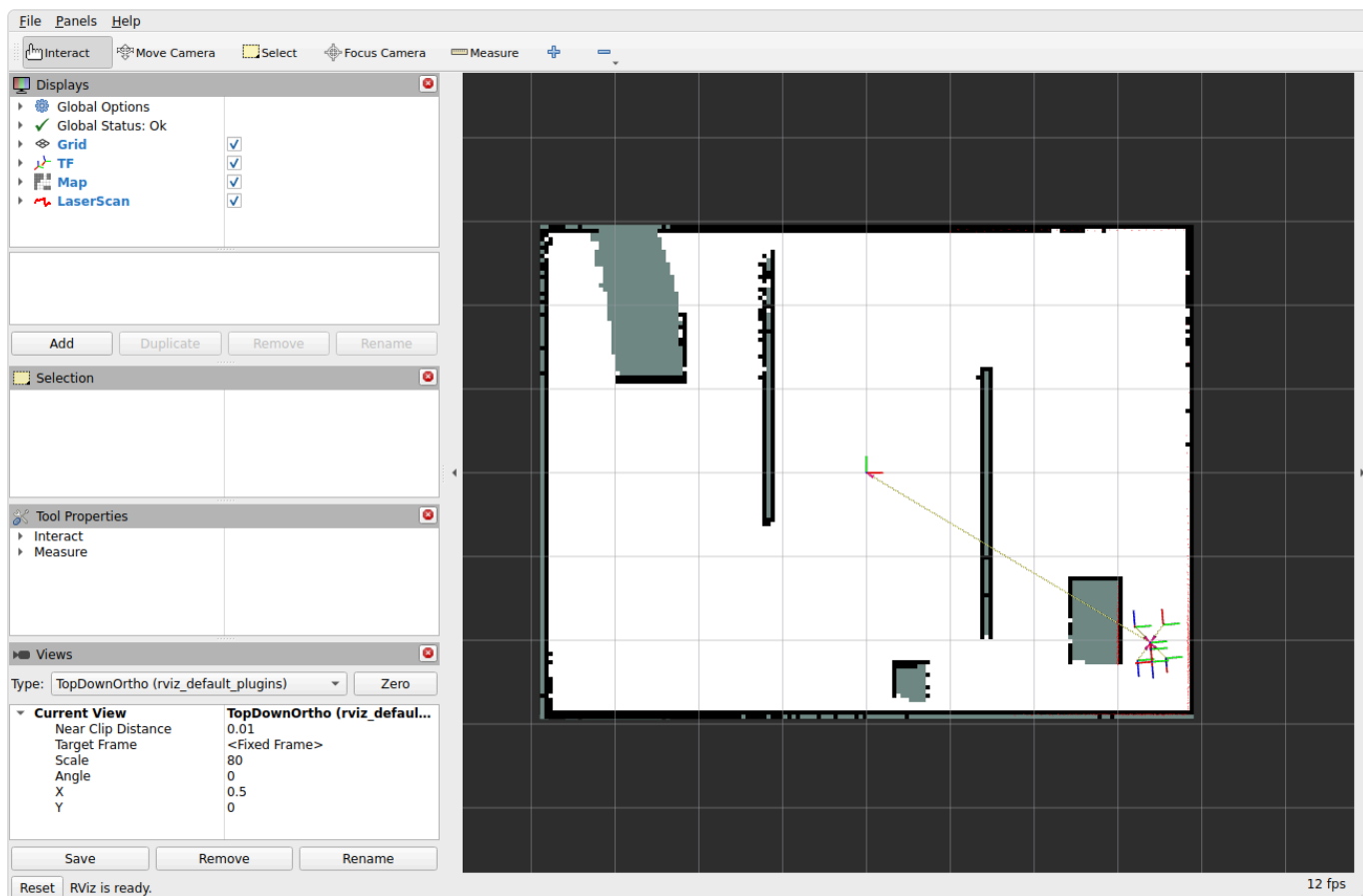
结束：在 launch 终端按 `Ctrl+C`，等待子进程退出。

5. LAB 2：半自动建图

```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch mecamind_bringup mapping.launch.py \
  auto_route:=true \
  map_save_path:=maps/mecamind_three_room_map \
  route_report_path:=maps/mecamind_three_room_route_report.json
```

`mapping_route_driver` 会按 `mecamind_mapping_route.yaml` 中的路点巡航（9 个路点：客厅入口 → 下方走廊 → 门厅 → 折返 → 右上走廊 → 卧室通道 → 卧室入口 → 卧室中心），结束后自动存图并写路线报告，默认再关闭整个 launch。

路线跑完后，RViz 俯视（视图切到 TopDownOrtho）应能看到完整闭合的三房间地图，机器人停在右下角最后一个路点：



查看报告：

```
python3 -m json.tool maps/mecamind_three_room_route_report.json | head -40
```

关注 `reached_waypoints` / `route_size`、`skipped_waypoints`、`auto_map_saved`。路点大量 `skipped` 时，地图往往不完整，需要重跑或改手动补扫。

实测参考（Gazebo 后端一次完整运行）：`reached_waypoints=9/9`、`skipped=0`、`auto_map_saved=true`，全程约 5~6 分钟（WSL2 下 Gazebo RTF < 1，比墙钟慢是正常的；路线驱动内部按仿真时间计超时，不会因此误判）。

6. LAB 3：轻量仿真建图（兜底）

```
ros2 launch mecamind_bringup mapping.launch.py \
  backend:=lite \
  use_gui:=false \
  with_rviz:=true \
  auto_route:=true \
  map_save_path:=maps/mecamind_lite_map
```

无显示器时可加 `with_rviz:=false`。接口与 Gazebo 相同，适合低配机与 CI。

7. LAB 4：地图质量分析与资产清单

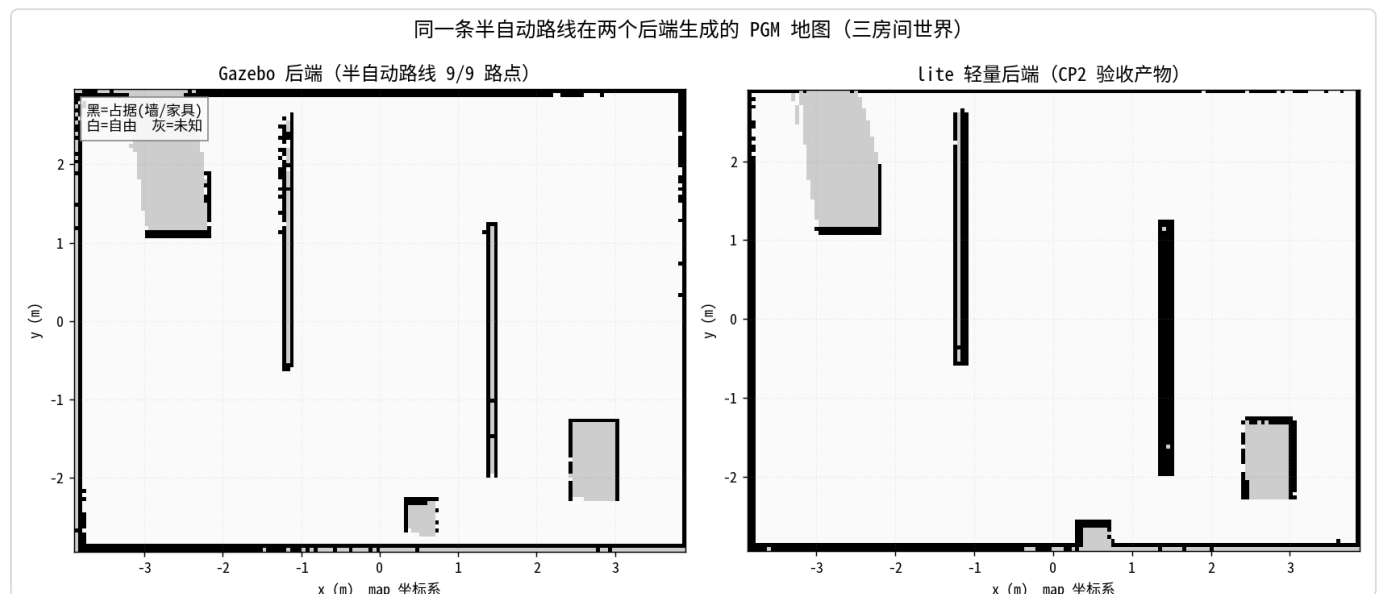
```
cd ~/mecamind_ros2
source /opt/ros/jazzy/setup.bash
source install/setup.bash

WORLD="$(ros2 pkg prefix mecamind_tools)/share/mecamind_tools/worlds/three_room_house.world"

ros2 run mecamind_tools mecamind_map_quality_analyzer \
  maps/mecamind_three_room_map.yaml \
  --world "$WORLD" --json | tee maps/mecamind_three_room_quality.json

ros2 run mecamind_tools mecamind_map_asset_manager maps \
  --world "$WORLD" \
  --output maps/mecamind_manifest.json
```

两个后端在同一条半自动路线下产出的 PGM 地图对比（黑=占据、白=自由、灰=未知；能看出四面外墙、两条内隔墙和几件家具的轮廓一致）：



合格示例特征（数值因后端略有差异）：

- coverage 较高（例如 > 0.85 ）。
- missing_walls 为空或很少。
- verdict 为 commercial_ready 或 navigation_ready_best_effort。

实测参考（Gazebo 后端半自动建图一次运行的质量报告）：

指标	实测值	解读
coverage	0.946	ROI 内 94.6% 的栅格已观测（自由或占据）
unknown_ratio	0.054	只剩 5.4% 未知，主要在家具背面的激光阴影
largest_free_component_ratio	0.9996	自由区几乎完全连通，房间没有被"假墙"隔断
wall_scores	北 0.94 / 南 1.0 / 东 1.0 / 西 1.0	四面外墙采样得分，越接近 1 越完整
missing_walls	空	没有整面缺失的墙
verdict	navigation_ready_best_effort	达到课堂验收线，可供下节课 Nav2 使用

若为 `needs_remap`：沿缺失墙边慢速再扫一遍，或重新半自动建图；建图前务必 `cleanup_ros2.sh`，避免多套仿真抢话题。

8. CP2 自动验收

```
cd ~/mecamind_ros2
bash scripts/cleanup_ros2.sh --all # 确认无残留后再跑
bash scripts/test_cp2.sh          # 默认 lite 后端
```

Gazebo headless:

```
MECAMIND_CP2_BACKEND=gazebo bash scripts/test_cp2.sh
```

脚本流程：半自动建图 → 检查 PGM/YAML/路线报告 → 质量分析 → 资产清单。成功标志：

```
MECAMIND_CP2_OK
```

产物示例：

```
maps/mecamind_cp2_lite_map.pgm
maps/mecamind_cp2_lite_map.yaml
maps/mecamind_cp2_lite_route_report.json
maps/mecamind_cp2_lite_quality.json
maps/mecamind_cp2_lite_manifest.json
```

9. 常见问题

9.1 RViz 里地图一直是空的

检查：

```
ros2 topic echo --once /clock
ros2 topic hz /scan
ros2 topic echo --once /map
ros2 run tf2_tools view_frames --ros-args -p use_sim_time:=true
```

确认 Fixed Frame 为 `map`，且存在 `map → odom → base_footprint`。

9.2 TF_OLD_DATA / 时间跳变

多半是残留的旧 Gazebo/SLAM 仍在发 TF。执行：

```
bash scripts/cleanup_ros2.sh --all
```

再只启动一套建图。

9.3 机器人不动或一直被刹停

确认遥控 remap 到 `/cmd_vel_nav`；查看安全门是否因前方过近而拦截：

```
ros2 topic echo /cmd_vel_nav
ros2 topic echo /cmd_vel
```

9.4 地图缺墙、覆盖率低

- 是否高速掠过、未正对墙面。
- 半自动是否大量 `skipped` 路点。
- 是否多套仿真同时运行导致激光/TF 混乱。
- 存图后可用质量分析定位 `missing_walls`。

9.5 `map_saver` 报没有地图

`/map` 多为 `transient local` QoS，且建图节点都在仿真时钟上。半自动路线驱动会缓存 `/map` 并直接写 PGM/YAML；手动存图请用：

```
ros2 run nav2_map_server map_saver_cli -f maps/mecamind_three_room_map \
--ros-args -p map_subscribe_transient_local:=true -p use_sim_time:=true
```

且 SLAM 节点必须仍在运行。轻量后端下若省略 `use_sim_time:=true`，常见报错是 `Failed to spin map subscription`；Gazebo 上有时仍能存图，但课堂统一加上更稳妥。

9.6 Gazebo 建图很慢

WSL2 下 RTF 常 < 1 。半自动超时可调环境变量（CP2 默认 600s）：

```
MECAMIND_CP2_TIMEOUT_SEC=900 MECAMIND_CP2_BACKEND=gazebo bash scripts/test_cp2.sh
```

10. 本节验收清单

- ☐ 能画出建图数据流（scan/odom/TF/clock $\rightarrow$ SLAM $\rightarrow$ map）。
- ☐ 能说明 `map` 与 `odom` 的区别。
- ☐ Gazebo 手动建图并在 RViz 看到 `/map` 增长。
- ☐ 保存完整的 PGM/YAML 地图对到 `maps/`。
- ☐ 半自动建图能写出路线报告。
- ☐ 质量分析 `verdict` 达到可导航级别。
- ☐ `scripts/test_cp2.sh`（lite）输出 `MECAMIND_CP2_OK`。
- ☐ （选做）Gazebo 后端 CP2 也通过。

11. 课后作业

必做：

- 提交一张建图过程 RViz 截图（含 Map + LaserScan）。
- 提交 `maps/` 下的一对 PGM/YAML（可自命名）。
- 提交对应的质量分析 JSON，并口头说明 `coverage` 与 `missing_walls`。
- 用文字说明：若关掉 `use_sim_time`，建图会出现什么现象。

选做：

- 对比同一路线在 `backend:=gazebo` 与 `backend:=lite` 下的质量报告差异。
- 调整 `mecamind_slam_toolbox_params.yaml` 中的 `resolution` 或 `minimum_travel_distance`，观察地图细腻度与建图速度变化（改后需重新 `build` 并重启 `launch`）。
- 查阅 Cartographer 或 RTAB-Map 文档，写 200 字以内的“为何本课不选它”的理由。

12. 下一次课预告

下一次课加载本节地图，启动 Nav2 (Map Server、AMCL、Planner、Controller)，完成命名目标导航与多点巡航。

附：本讲义所有截图（ docs/images/10.3.2/ ）均来自当前代码在 WSL2 + ROS 2 Jazzy + Gazebo Harmonic 上的一次真实运行：半自动路线 9/9 路点全部到达、自动存图成功、质量分析 verdict 为 navigation_ready_best_effort 。复现命令即第 5 节 LAB 2；节点图由 rqt_graph 数据导出，TF 树由 tf2_tools view_frames 导出。